US Patent & Trademark Office
Patent Public Search | Text View

United States Patent	12399687
Kind Code	B2
Date of Patent	August 26, 2025
Inventor(s)	Guttridge; Francis James Alexander et al.

Generating software architecture from conversation

Abstract

An example operation may include one or more of receiving and recording electronic communications that occur between users of an organization within a data store, receiving an input via a user interface, retrieving the electronic communications of the organization and software architecture documents from the data store in response to receipt of the input, generating a diagram of a software architecture of the organization based on execution of a generative artificial intelligence (GenAI) model on the electronic communications of the organization and the software architecture documents, and displaying the diagram of the software architecture via a user interface.

Inventors: **Guttridge; Francis James Alexander (Vaughan, CA), D'Agostino; Dino Paul (Richmond Hill, CA), Pratten; A Warren (London, CA), Mohammed; Shahzad (Toronto, CA), Chikkalavalasa; Arvind (Milton, CA), Nawaz; Waqas (Waterdown, CA)**

Applicant: **The Toronto-Dominion Bank** (Toronto, CA)

Family ID: **1000008781766**

Assignee: **The Toronto-Dominion Bank (Toronto, CA)**

Appl. No.: **18/458972**

Filed: **August 30, 2023**

Prior Publication Data

Document Identifier	Publication Date
US 20250077187 A1	Mar. 06, 2025

Publication Classification

Int. Cl.: **G06F8/10** (20180101)

U.S. Cl.:

CPC **G06F8/10** (20130101);

Field of Classification Search

CPC: G06F (8/10)

USPC: 717/104-120; 706/25-45

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
7120643	12/2005	Dill	N/A	N/A
7222138	12/2006	Fomenko	N/A	N/A
7376645	12/2007	Bernard	N/A	N/A
7716254	12/2009	Sarkar et al.	N/A	N/A
7880749	12/2010	Favart et al.	N/A	N/A
8775462	12/2013	Coldicott et al.	N/A	N/A
8793151	12/2013	DelZoppo	705/7.28	G06Q 10/0635
9342279	12/2015	Zhang et al.	N/A	N/A
9372669	12/2015	Jalaldeen et al.	N/A	N/A
9389851	12/2015	Nelson et al.	N/A	N/A
9477444	12/2015	Shen et al.	N/A	N/A
10289615	12/2018	Nijor et al.	N/A	N/A
10303533	12/2018	Panov et al.	N/A	N/A
10339931	12/2018	Tseretopoulos et al.	N/A	N/A
10360303	12/2018	Volkovs et al.	N/A	N/A
10379994	12/2018	Giles et al.	N/A	N/A
10460748	12/2018	Tseretopoulos et al.	N/A	N/A
10482675	12/2018	Sutter et al.	N/A	N/A
10503486	12/2018	Dimitrov et al.	N/A	N/A
10572473	12/2019	Allen et al.	N/A	N/A
10659400	12/2019	Moon et al.	N/A	N/A
10706635	12/2019	Sutter et al.	N/A	N/A
10728259	12/2019	McCarter et al.	N/A	N/A
10756982	12/2019	Bai	N/A	N/A
10776619	12/2019	Collinson et al.	N/A	N/A
10824941	12/2019	Volkovs et al.	N/A	N/A
10878816	12/2019	Tseretopoulos et al.	N/A	N/A
10902220	12/2020	Lozon et al.	N/A	N/A
10922665	12/2020	Miller et al.	N/A	N/A
10943605	12/2020	Tseretopoulos et al.	N/A	N/A
10949337	12/2020	Yalla	N/A	G06F 11/3688
10997227	12/2020	Agrawal et al.	N/A	N/A
11004187	12/2020	Kuruvilla et al.	N/A	N/A
11017028	12/2020	Dunjic et al.	N/A	N/A
11017156	12/2020	Hwang	N/A	N/A

11023689	12/2020	Sippel et al.	N/A	N/A
11030068	12/2020	Agarwal et al.	N/A	N/A
11030415	12/2020	Volkovs et al.	N/A	N/A
11061638	12/2020	Lam	N/A	N/A
11070448	12/2020	Miller et al.	N/A	N/A
11086859	12/2020	Nijor et al.	N/A	N/A
11087314	12/2020	Gandhi et al.	N/A	N/A
11100168	12/2020	Miller et al.	N/A	N/A
11144921	12/2020	Dunjic et al.	N/A	N/A
11144998	12/2020	Kuruvilla et al.	N/A	N/A
11182860	12/2020	Kuruvilla et al.	N/A	N/A
11194688	12/2020	Featonby et al.	N/A	N/A
11200328	12/2020	Shpurov et al.	N/A	N/A
11200411	12/2020	Rizvi et al.	N/A	N/A
11222286	12/2021	Choe et al.	N/A	N/A
11237802	12/2021	Krishnamoorthy et al.	N/A	N/A
11250063	12/2021	Krasadakis	N/A	N/A
11270206	12/2021	Kursun	N/A	G06N 3/082
11276257	12/2021	Moghtadai et al.	N/A	N/A
11314620	12/2021	Lin et al.	N/A	N/A
11334574	12/2021	Caputo et al.	N/A	N/A
11354442	12/2021	Haldenby et al.	N/A	N/A
11361566	12/2021	Collinson et al.	N/A	N/A
11373229	12/2021	Tseretopoulos et al.	N/A	N/A
11385892	12/2021	Zhang	N/A	N/A
11392776	12/2021	Lozon et al.	N/A	N/A
11393020	12/2021	Mathew et al.	N/A	N/A
11394668	12/2021	Subbunarayanan et al.	N/A	N/A
11397765	12/2021	Volkovs et al.	N/A	N/A
11409811	12/2021	D'Agostino	N/A	N/A
11411734	12/2021	Shpurov et al.	N/A	N/A
11443082	12/2021	Janarthanam et al.	N/A	N/A
11469878	12/2021	Shpurov et al.	N/A	N/A
11470091	12/2021	McCarter et al.	N/A	N/A
11475059	12/2021	Liu et al.	N/A	N/A
11475251	12/2021	Morin et al.	N/A	N/A
11477265	12/2021	McPhee et al.	N/A	N/A
11507622	12/2021	Grebenisan et al.	N/A	N/A
11507868	12/2021	Kwong et al.	N/A	N/A
11537492	12/2021	Agarwal et al.	N/A	N/A
11550652	12/2022	Arora et al.	N/A	N/A
11580762	12/2022	Rizvi et al.	N/A	N/A
11586491	12/2022	Banerjee et al.	N/A	N/A
11593461	12/2022	Polisetty et al.	N/A	N/A
11599444	12/2022	Lin et al.	N/A	N/A
11604899	12/2022	Haldenby et al.	N/A	N/A
11620741	12/2022	Kuruvilla et al.	N/A	N/A
11632311	12/2022	Miller et al.	N/A	N/A
11663488	12/2022	Volkovs et al.	N/A	N/A
11669436	12/2022	Ingram	N/A	N/A

11675654	12/2022	Pudipeddi et al.	N/A	N/A
11676016	12/2022	Baijal et al.	N/A	N/A
11687995	12/2022	Tseretopoulos et al.	N/A	N/A
11689484	12/2022	Moon et al.	N/A	N/A
11704782	12/2022	Wakim et al.	N/A	N/A
11741305	12/2022	Skaljin et al.	N/A	N/A
11743210	12/2022	Moon et al.	N/A	N/A
11748400	12/2022	Volkovs et al.	N/A	N/A
11748555	12/2022	Tran	715/202	G06F 40/137
11782935	12/2022	Caputo et al.	N/A	N/A
11789909	12/2022	Grebenisan et al.	N/A	N/A
11790012	12/2022	D'Agostino	N/A	N/A
11790354	12/2022	Gandhi et al.	N/A	N/A
11809486	12/2022	Liu et al.	N/A	N/A
11809577	12/2022	Begg et al.	N/A	N/A
11830463	12/2022	Kurek	N/A	G10H 1/0025
11842252	12/2022	Kuang et al.	N/A	N/A
11868672	12/2023	Dehkordi	N/A	G06F 9/451
11886764	12/2023	Lam	N/A	N/A
11928112	12/2023	Dunjic et al.	N/A	N/A
11941525	12/2023	Morin et al.	N/A	N/A
11966491	12/2023	D'Agostino	N/A	N/A
11978085	12/2023	Rai et al.	N/A	N/A
11978090	12/2023	Navarro et al.	N/A	N/A
11995121	12/2023	Volkovs et al.	N/A	N/A
12008315	12/2023	Miller et al.	N/A	N/A
12021874	12/2023	Dunjic et al.	N/A	N/A
12039535	12/2023	Dunjic et al.	N/A	N/A
12052363	12/2023	Shpurov et al.	N/A	N/A
12061652	12/2023	Miller et al.	N/A	N/A
12067130	12/2023	Shpurov et al.	N/A	N/A
12067580	12/2023	Jeske et al.	N/A	N/A
12079351	12/2023	Begg et al.	N/A	N/A
12106220	12/2023	Volkovs et al.	N/A	N/A
12111793	12/2023	Grebenisan et al.	N/A	N/A
12124925	12/2023	Rho et al.	N/A	N/A
12136079	12/2023	Jones et al.	N/A	N/A
12158929	12/2023	Huang	N/A	H04L 9/0643
12164751	12/2023	Harper	N/A	G06F 3/04883
12182263	12/2023	Sinn	N/A	G06F 21/554
12182678	12/2023	Poulis	N/A	G06N 3/045
12204323	12/2024	Malviya	N/A	G05B 23/0216
12223456	12/2024	Manohar	N/A	G06F 40/30
2007/0061354	12/2006	Sarkar et al.	N/A	N/A
2009/0064205	12/2008	Fay et al.	N/A	N/A
2015/0082272	12/2014	Jalaldeen et al.	N/A	N/A
2018/0150378	12/2017	Gopalswamy et al.	N/A	N/A
2018/0268818	12/2017	Schoenmackers et al.	N/A	N/A
2020/0058068	12/2019	Gandhi et al.	N/A	N/A
2021/0027160	12/2020	Volkovs et al.	N/A	N/A

2021/0037040	12/2020	Aleks et al.	N/A	N/A
2021/0049007	12/2020	Nelluri et al.	N/A	N/A
2021/0117893	12/2020	Sohum et al.	N/A	N/A
2021/0232950	12/2020	Kono	N/A	N/A
2021/0264461	12/2020	Fam	N/A	N/A
2021/0334700	12/2020	Nagaraja	N/A	N/A
2021/0407016	12/2020	Kuruvilla et al.	N/A	N/A
2022/0035856	12/2021	Gilder et al.	N/A	N/A
2022/0058489	12/2021	Volkovs et al.	N/A	N/A
2022/0107980	12/2021	Lisuk et al.	N/A	N/A
2022/0108069	12/2021	Lee	N/A	N/A
2022/0157094	12/2021	Moghtadai et al.	N/A	N/A
2022/0172083	12/2021	Wu et al.	N/A	N/A
2022/0188705	12/2021	Davoodi et al.	N/A	N/A
2022/0198411	12/2021	Jones et al.	N/A	N/A
2022/0198445	12/2021	Jones et al.	N/A	N/A
2022/0207295	12/2021	Stanevich et al.	N/A	N/A
2022/0207430	12/2021	Dickie et al.	N/A	N/A
2022/0207432	12/2021	Whelan et al.	N/A	N/A
2022/0207606	12/2021	Dickie et al.	N/A	N/A
2022/0270155	12/2021	Volkovs et al.	N/A	N/A
2022/0277213	12/2021	Braviner et al.	N/A	N/A
2022/0277227	12/2021	Yu et al.	N/A	N/A
2022/0277323	12/2021	Whelan et al.	N/A	N/A
2022/0300903	12/2021	Huang et al.	N/A	N/A
2022/0309573	12/2021	Mathew et al.	N/A	N/A
2022/0318500	12/2021	Prasad et al.	N/A	N/A
2022/0318573	12/2021	Smith et al.	N/A	N/A
2022/0318617	12/2021	Wong et al.	N/A	N/A
2022/0318682	12/2021	Sawaf et al.	N/A	N/A
2022/0318683	12/2021	Sawaf et al.	N/A	N/A
2022/0327397	12/2021	Braviner et al.	N/A	N/A
2022/0327430	12/2021	Zuberi et al.	N/A	N/A
2022/0327431	12/2021	Braviner et al.	N/A	N/A
2022/0327432	12/2021	Gutierrez Bugarin et al.	N/A	N/A
2022/0327625	12/2021	Leung et al.	N/A	N/A
2022/0335718	12/2021	Ma et al.	N/A	N/A
2022/0343422	12/2021	Zuberi et al.	N/A	N/A
2022/0383313	12/2021	Jones et al.	N/A	N/A
2022/0414495	12/2021	Stanevich et al.	N/A	N/A
2023/0006809	12/2022	Shpurov et al.	N/A	N/A
2023/0007075	12/2022	McPhee et al.	N/A	N/A
2023/0011451	12/2022	Lu	N/A	N/A
2023/0048437	12/2022	Karbasi et al.	N/A	N/A
2023/0083899	12/2022	Gandouet et al.	N/A	N/A
2023/0086653	12/2022	Zykh et al.	N/A	N/A
2023/0107703	12/2022	Zhang et al.	N/A	N/A
2023/0113752	12/2022	Jorlett et al.	N/A	N/A
2023/0119108	12/2022	Volkovs et al.	N/A	N/A

2023/0131935	12/2022	Volkovs et al.	N/A	N/A
2023/0153461	12/2022	Kalra et al.	N/A	N/A
2023/0195734	12/2022	Cashion et al.	N/A	N/A
2023/0196406	12/2022	Gandouet et al.	N/A	N/A
2023/0244917	12/2022	Loaiza Ganem et al.	N/A	N/A
2023/0244962	12/2022	Volkovs et al.	N/A	N/A
2023/0252301	12/2022	Volkovs et al.	N/A	N/A
2023/0259883	12/2022	Misler et al.	N/A	N/A
2023/0267367	12/2022	Volkovs et al.	N/A	N/A
2023/0267475	12/2022	Navarro et al.	N/A	N/A
2023/0318994	12/2022	Moon et al.	N/A	N/A
2023/0351116	12/2022	Skaljin et al.	N/A	N/A
2023/0368048	12/2022	Yang et al.	N/A	N/A
2023/0377047	12/2022	Bouëtté et al.	N/A	N/A
2023/0385693	12/2022	Cresswell et al.	N/A	N/A
2023/0385694	12/2022	Cresswell et al.	N/A	N/A
2023/0386190	12/2022	Cresswell et al.	N/A	N/A
2023/0401192	12/2022	Yang et al.	N/A	N/A
2023/0401553	12/2022	Navarro et al.	N/A	N/A
2023/0419302	12/2022	Navarro et al.	N/A	N/A
2023/0419402	12/2022	Ghelichi et al.	N/A	N/A
2024/0020534	12/2023	Perez Vallejo et al.	N/A	N/A
2024/0062117	12/2023	Kuang et al.	N/A	N/A
2024/0106851	12/2023	Kennedy et al.	N/A	N/A
2024/0119346	12/2023	Chang et al.	N/A	N/A
2024/0127036	12/2023	Zuberi et al.	N/A	N/A
2024/0127214	12/2023	Wander et al.	N/A	N/A
2024/0193562	12/2023	Pratten et al.	N/A	N/A
2024/0202756	12/2023	Karl et al.	N/A	N/A
2024/0211732	12/2023	Wander et al.	N/A	N/A
2024/0212049	12/2023	Ghelichi et al.	N/A	N/A
2024/0220653	12/2023	D'Agostino	N/A	N/A
2024/0232614	12/2023	Esmaeili et al.	N/A	N/A
2024/0232950	12/2023	Navarro et al.	N/A	N/A
2024/0256903	12/2023	Ens et al.	N/A	N/A
2024/0256904	12/2023	Leung et al.	N/A	N/A
2024/0256968	12/2023	Hosseinzadeh et al.	N/A	N/A
2024/0281467	12/2023	Volkovs et al.	N/A	N/A
2024/0281808	12/2023	Vouitsis et al.	N/A	N/A
2024/0281818	12/2023	Golestan Irani et al.	N/A	N/A
2024/0289645	12/2023	Makhijani et al.	N/A	N/A
2024/0289876	12/2023	Mathew et al.	N/A	N/A
2024/0303551	12/2023	Li et al.	N/A	N/A
2024/0304182	12/2023	Hamilton et al.	N/A	N/A
2024/0330772	12/2023	Cresswell et al.	N/A	N/A
2024/0338520	12/2023	Misler et al.	N/A	N/A
2024/0346338	12/2023	Desai et al.	N/A	N/A
2024/0370880	12/2023	Jeske et al.	N/A	N/A
2024/0370881	12/2023	Jeske et al.	N/A	N/A

2024/0430363	12/2023	Neate	N/A	H04M 3/53366

OTHER PUBLICATIONS

Santos et al, “Impacts of the Usage of Generative Artificial Intelligence on Software Development Process”, ACM, pp. 1-9 (Year: 2024). cited by examiner
Heidrich et al, “Visualizing Source Code as Comics Using Generative AI”, IEEE, pp. 1-5 (Year: 2023). cited by examiner
Arun, “A Software Architectural Model for Generative Aftifical Intelligence with Reinforcement Learning”, IEEE, pp. 1-6 (Year: 2024). cited by examiner
He et al, “Securing Federated Diffusion Model With Dynamic Quantization for Generative AI Services in Multiple-Access Artifical Intelligence of Things”, IEEE, pp. 1-14 (Year: 2024). cited by examiner
Zhang et al, “Application Research on Artifical Intelligence Generated Content in Architectural Design”, ACM, pp. 1-10 (Year: 2024). cited by examiner

---

Primary Examiner: Khatri; Anil

Background/Summary

BACKGROUND
(1) For large companies and organizations, the number of software architects can be in the dozens or even hundreds. The architects often work inside a siloed environment outside of a larger software architecture ecosystem of the organization. For example, a first software architect may organize and manage an email software system, while a second software architect may manage a database software system. In this example, these architects may be unaware of the software architecture of the other and of the others in the larger ecosystem.
(2) Furthermore, external audits are often performed by external users of an organization that verify licensing rights and identify compliance gaps. These audits are aided significantly by drawings of the architecture. However, these diagrams are often not made by the architect or not completed with enough current information to fully understand the software system as it currently is in real-time.
SUMMARY
(3) One example embodiment provides an apparatus that may include a data store configured to store software architecture diagrams, and a processor configured to receive runtime data from a plurality of different software systems within a software architecture, the runtime data comprising descriptions of events that occur during runtime between the different software systems of the software architecture, generate a diagram of the software architecture based on execution of a multi-modal generative artificial intelligence (GenAI) model on the runtime data and the stored software architecture diagrams in the data store, and display the diagram of the software architecture via a user interface.
(4) Another example embodiment provides a method that includes one or more of storing software architecture diagrams in a data store, receiving runtime data from a plurality of different software systems within a software architecture, the runtime data comprising descriptions of events that occur during runtime between the different software systems of the software architecture, generating a diagram of the software architecture based on execution of a multi-modal generative artificial intelligence (GenAI) model on the runtime data and the software architecture diagrams stored in the data store, and displaying the diagram of the software architecture via a user interface.

(5) A further example embodiment provides a computer-readable medium comprising instructions, that when read by a processor, cause the processor to perform one or more of storing software architecture diagrams in a data store, receiving runtime data from a plurality of different software systems within a software architecture, the runtime data comprising descriptions of events that occur during runtime between the different software systems of the software architecture, generating a diagram of the software architecture based on execution of a multi-modal generative artificial intelligence (GenAI) model on the runtime data and the software architecture diagrams stored in the data store, and displaying the diagram of the software architecture via a user interface.
(6) One example embodiment provides an apparatus that may include a data store comprising descriptions and diagrams of a software architecture, and a processor configured to train a generative artificial intelligence (GenAI) model via execution of the GenAI model on descriptions and diagrams of a software architecture, receive a plurality of different architecture views from a plurality of different domains of the software architecture, generate an architecture diagram of the plurality of domains of the software architecture in combination based on execution of the GenAI model on the plurality of different architecture views from the plurality of different domains, and display the architecture diagram of the plurality of domains of the software architecture via a user interface.
(7) Another example embodiment provides a method that includes one or more of training a generative artificial intelligence (GenAI) model via execution of the GenAI model on descriptions and diagrams of a software architecture, receiving a plurality of different architecture views from a plurality of different domains of the software architecture, generating an architecture diagram of the plurality of domains of the software architecture in combination based on execution of the GenAI model on the plurality of different architecture views from the plurality of different domains, and displaying the architecture diagram of the plurality of domains of the software architecture via a user interface.
(8) A further example embodiment provides a computer-readable medium comprising instructions, that when read by a processor, cause the processor to perform one or more of training a generative artificial intelligence (GenAI) model via execution of the GenAI model on descriptions and diagrams of a software architecture, receiving a plurality of different architecture views from a plurality of different domains of the software architecture, generating an architecture diagram of the plurality of domains of the software architecture in combination based on execution of the GenAI model on the plurality of different architecture views from the plurality of different domains, and displaying the architecture diagram of the plurality of domains of the software architecture via a user interface.
(9) One example embodiment provides an apparatus that may include a processor configured to receive a plurality of architecture documents of a plurality of different domains of a software architecture, identify a missing component that is missing from within the software architecture between a first domain and a second domain among the plurality of domains based on execution of a generative artificial intelligence (GenAI) model on the plurality of architecture documents of the software architecture, generate a recommended modification to the software architecture based on the identified missing component, and display the recommended modification via a user interface.
(10) Another example embodiment provides a method that includes one or more of receiving a plurality of architecture documents of a plurality of different domains of a software architecture, identifying a missing component that is missing from within the software architecture between a first domain and a second domain among the plurality of domains based on execution of a generative artificial intelligence (GenAI) model based on the plurality of architecture documents of the software architecture, generating a recommended modification to the software architecture based on the identified missing component, and displaying the recommended modification via a user interface.
(11) A further example embodiment provides a computer-readable medium comprising instructions,

that when read by a processor, cause the processor to perform one or more of receiving a plurality of architecture documents of a plurality of different domains of a software architecture, identifying a missing component that is missing from within the software architecture between a first domain and a second domain among the plurality of domains based on execution of a generative artificial intelligence (GenAI) model based on the plurality of architecture documents of the software architecture, generating a recommended modification to the software architecture based on the identified missing component, and displaying the recommended modification via a user interface.

(12) A further example embodiment provides an apparatus that may include a processor configured to train a generative artificial intelligence (GenAI) model based on architecture diagrams of a software architecture and descriptions of the architecture diagrams, display one or more prompts on a user interface, receive one or more natural language responses associated with the software architecture in response to the one or more prompts, generate a text-based response to the natural language query submitted via the user interface based on execution of the GenAI model on the one or more prompts and the one or more natural language responses associated with the software architecture, and display the text-based response via the user interface.

(13) A further example embodiment provides a method that includes one or more of training a generative artificial intelligence (GenAI) model based on architecture diagrams of a software architecture and descriptions of the architecture diagrams, displaying one or more prompts on a user interface, receiving one or more natural language responses associated with the software architecture in response to the one or more prompts, generating a text-based response to the natural language query submitted via the user interface based on execution of the GenAI model on the one or more prompts and the one or more natural language responses associated with the software architecture, and displaying the text-based response via the user interface.

(14) A further example embodiment provides a computer-readable medium comprising instructions, that when read by a processor, cause the processor to perform one or more of training a generative artificial intelligence (GenAI) model based on architecture diagrams of a software architecture and descriptions of the architecture diagrams, displaying one or more prompts on a user interface, receiving one or more natural language responses associated with the software architecture in response to the one or more prompts, generating a text-based response to the natural language query submitted via the user interface based on execution of the GenAI model on the one or more prompts and the one or more natural language responses associated with the software architecture, and displaying the text-based response via the user interface.

(15) A further example embodiment provides an apparatus that may include a data store comprising software architecture documents, and a processor configured to receive and record electronic communications that occur between users of an organization within the data store, receive an input via a user interface, retrieve the electronic communications of the organization and software architecture documents from the data store in response to receipt of the input, generate a diagram of a software architecture of the organization based on execution of a generative artificial intelligence (GenAI) model on the electronic communications of the organization and the software architecture documents, and display the diagram of the software architecture via a user interface.

(16) A further example embodiment provides a method that includes one or more of receiving and recording electronic communications that occur between users of an organization within a data store, receiving an input via a user interface, retrieving the electronic communications of the organization and software architecture documents from the data store in response to receipt of the input, generating a diagram of a software architecture of the organization based on execution of a generative artificial intelligence (GenAI) model on the electronic communications of the organization and the software architecture documents, and displaying the diagram of the software architecture via a user interface.

(17) A further example embodiment provides a computer-readable medium comprising instructions, that when read by a processor, cause the processor to perform one or more of

receiving and recording electronic communications that occur between users of an organization within a data store, receiving an input via a user interface, retrieving the electronic communications of the organization and software architecture documents from the data store in response to receipt of the input, generating a diagram of a software architecture of the organization based on execution of a generative artificial intelligence (GenAI) model on the electronic communications of the organization and the software architecture documents, and displaying the diagram of the software architecture via a user interface.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) FIG. **1**A is a diagram illustrating a generative artificial intelligence (GenAI) computing environment for generating architecture diagrams according to example embodiments.

(2) FIG. **1**B is a diagram illustrating an example of a generative architecture diagram according to example embodiments.

(3) FIG. **2** is a diagram illustrating a process of executing a machine-learning model on input content according to example embodiments.

(4) FIGS. **3**A-**3**C are diagrams illustrating processes for training a machine learning model according to example embodiments.

(5) FIG. **4** is a diagram illustrating a process of prompting a GenAI model to generate an architecture diagram according to example embodiments.

(6) FIGS. **5**A-**5**D are diagrams illustrating a process of generating a complex architecture diagram from a plurality of architecture views according to example embodiments.

(7) FIG. **6** is a diagram illustrating an example of a multi-modal generative artificial intelligence model according to example embodiments.

(8) FIGS. **7**A-**7**B are diagrams illustrating a process of auditing a software architecture via a GenAI model according to example embodiments.

(9) FIG. **8** is a diagram illustrating a process of building a software architecture diagram based on conversation data according to example embodiments.

(10) FIG. **9** is a diagram illustrating a process of generating a software architecture diagram using runtime data from systems of the software architecture according to example embodiments.

(11) FIG. **10**A is a diagram illustrating a method of generating a real-time diagram of a software architecture according to example embodiments.

(12) FIG. **10**B is a diagram illustrating a method of generating a multi-domain architecture diagram from multiple domain views according to example embodiments.

(13) FIG. **10**C is a diagram illustrating a method of identifying gaps in a software architecture according to example embodiments.

(14) FIG. **10**D is a diagram illustrating a method of auditing a software architecture via a GenAI model according to example embodiments.

(15) FIG. **10**E is a diagram illustrating a method of generating a diagram of a software architecture based on conversation data according to example embodiments.

(16) FIG. **11** is a diagram illustrating a computing system that may be used in any of the example embodiments described herein.

DETAILED DESCRIPTION

(17) It is to be understood that although this disclosure includes a detailed description of cloud computing, implementation of the teachings recited herein is not limited to a cloud computing environment. Rather, embodiments of the instant solution are capable of being implemented in conjunction with any other type of computing environment now known or later developed.

(18) The example embodiments are directed to a platform that generates diagrams of software

architecture based on generative artificial intelligence. In some embodiments, a generative artificial intelligence (GenAI) model may be trained to understand software architecture based on a large corpus of architecture documents and architecture descriptions. Through the training process, the GenAI model may learn a correlation between text (e.g., words) and software architecture components. Furthermore, the GenAI model may also generate architecture diagrams.

(19) According to various embodiments, the GenAI model may be a large language model (LLM), such as a multimodal large language model. As another example, the GenAI model may be a transformer neural network ("transformer") or the like. The GenAI model is capable of understanding connections between text and components (e.g., boxes, lines, arrows, software architecture, etc.) within software architecture drawings. For example, the GenAI model may include libraries and deep learning frameworks that enable the GenAI model to create realistic diagrams based on text inputs.

(20) By creating software architecture diagrams from text, the GenAI model can relieve a user from having to generate such diagrams. Furthermore, the GenAI model described herein can learn software architecture and provide recommendations to the architects of the software system. The recommendations can include recommendations for filling in software components that are not included within the software architecture.

(21) Furthermore, the GenAI model described herein can receive diagrams as input, such as partial diagrams/views of software architecture from a different respective domain and can generate a "complete" diagram of the software architecture of a plurality of different domains based on the partial diagrams. Here, the GenAI model may "fit" together the pieces of the software architecture and identify missing components that are needed to help the pieces of the software architecture co-exist.

(22) FIG. **1**A illustrates a GenAI computing environment for generating architecture diagrams according to example embodiments and FIG. **1**B illustrates an example of a generative architecture diagram according to example embodiments. For example, FIG. **1**A illustrates a process **100** of a host platform **120**, such as a cloud platform, web server, etc., interacting with a user device **110**, such as a mobile device, a computer, a laptop, or the like. As just one example, the host platform **120** may host a software application **122** that is accessed by the user device **110** over a computer network such as the Internet. As an example, the software application **122** may be a mobile application that includes a front-end that is installed on the user device **110** and a back-end that is installed on the host platform **120**. As another example, the software application **122** may be a progressive web application (PWA) that is hosted by the host platform **120** and made accessible to the user device **110** via an address on the web.

(23) In the example embodiments, the host platform **120** may include one or more generative artificial intelligence (GenAI) models, including GenAI model **124**, which can prompt a user for information (e.g., images, text, etc.) and generate software architecture diagrams based on responses to the prompts. The host platform **120** may also include one or more additional models, including one or more machine learning models, one or more artificial intelligence (AI) models, one or more additional GenAI models, and the like. The models, including the GenAI model **124**, may be held by the host platform **120** within a model repository (not shown).

(24) In the example embodiments, the GenAI model **124** may be trained based on software architecture documents and descriptions that the GenAI model can use to learn connections between text and diagram pieces. For example, the software architecture documents may be stored within a data store **126** on the host platform. In addition, the GenAI model **124** may learn and may also receive as input runtime data of the software architecture from a second data store **128** of the host platform **120**. Here, the runtime data may include runtime data/flow between the software components of the software architecture and may be used by the system to generate a "real-time" view of the software architecture.

(25) As an example, the runtime data from the second data store **128** may include API calls,

database queries, data transfers, method calls, and the like, which can be used by the GenAI model **124** to diagram the software architecture. In this example, the GenAI model **124** may identify the current components that are running within the software architecture based on the runtime data. This method is more accurate than a user "remembering" the content within the software architecture because it's based on actual data that is generated by the software systems in the software architecture. The GenAI model **124** can understand connections between entries in the runtime data/log data and diagram components.

(26) The data store **126** and/or the second data store **128** may be accessed via one or more application programming interfaces (APIs). Although not shown, the host platform **120** may also access one or more external systems (e.g., databases, websites, etc.) over a computer network and collect/retrieve data from the one or more external systems, including user data.

(27) In the example of FIG. **1**A, a user has entered a text-based input into a user interface **112** on the user device **110**, which is sent to the software application **122** on the host platform **120** over the network. The text may be entered in response to a prompt on the user interface **112** that is generated by the GenAI model **124**. However, a prompt is not required. In other words, the user could directly query the software application **122** for information about the software architecture. The software application **122** inputs the text input into the GenAI model **124**, which generates an architecture diagram **130** in response to the text input. In this example, the text input may include a description (text) of a software architecture system or component, and the GenAI model **124** may convert the text into a diagram that shows the corresponding software architecture in diagram format. The architecture diagram **130** may be submitted to the user device **110** and displayed on the user interface **112**.

(28) Referring now to FIG. **1**B, the architecture diagram **130** output by the GenAI model **124** in FIG. **1**A is shown in more detail. In this example, the architecture diagram includes boxes, directional lines, arrows, text, and the like, which can be used to identify the software architecture of a software system. In this example, a first software system is shown inside a box **132** within the architecture diagram, and an edge **134** between the box **132** and a box **136** corresponds to a second software system is also shown. In some embodiments, the edge **134** may include annotations that identify the interaction between the two components, such as the method, function, etc. Furthermore, a customer **131** is shown interacting with different components within the software architecture.

(29) The GenAI model **124** is trained to generate diagrams of software architecture from text. For example, the model may be embedded with mappings between software architecture components and text descriptions. As an example, the GenAI model **124** may be a multi-modal large language model (LLM). In this example, a first modality of the GenAI model **124** may receive text, images, etc., and convert it into a diagram, such as a UML diagram, that includes both text and graphics (images) of the software architecture. The diagram can then be converted into an architecture diagram (e.g., as shown in FIG. **1**B) based on a second modality.

(30) FIG. **2** illustrates a process **200** of executing a model **224** on input content according to example embodiments. As an example, the model **224** may be the GenAI model **124** described with respect to FIG. **1**A, however, embodiments are not limited thereto. Referring to FIG. **2**, a software application **210** may request execution of model **224** by submitting a request to the host platform **220**. In response, an AI engine **222** may receive the request and trigger the model **224** to execute within a runtime environment of the host platform **220**.

(31) In FIG. **2**, the AI engine **222** may control access to models that are stored within the model repository **223**. For example, the models may include GenAI models, AI models, machine learning models, neural networks, and/or the like. The software application **210** may trigger execution of the model **224** from the model repository **223** via submission of a call to an API **221** (application programming interface) of the AI engine **222**. The request may include an identifier of the model **224**, such as a unique ID assigned by the host platform **220**, a payload of data (e.g., to be input to

the model during execution), and the like. The AI engine **222** may retrieve the model **224** from the model repository **223** in response and deploy the model **224** within a live runtime environment. After the model is deployed, the AI engine **222** may execute the running instance of the model **224** on the payload of data and return a result of the execution to the software application **210**.

(32) In some embodiments, the payload of data may be a format that is not capable of being input to the model **224** nor read by a computer processor. For example, the payload of data may be in text format, image format, audio format, and the like. In response, the AI engine **222** may convert the payload of data into a format that is readable by the model **224**, such as a vector or other encoding. The vector may then be input to the model **224**.

(33) In some embodiments, the software application **210** may display a user interface that enables a user thereof to provide feedback from the output provided by the model **224**. For example, a user may input a confirmation that the predicted image of a goal generated by a GenAI model is correct or is liked. This information may be added to the results of execution and stored within a runtime log **225**. The runtime log **225** may include an identifier of the input, an identifier of the output, an identifier of the model used, and feedback from the recipient. This information may be used to subsequently re-train the model.

(34) FIG. **3**A illustrates a process **300**A of training a GenAI model **322** according to example embodiments. However, it should be appreciated that the process **300**A, shown in FIG. **3**A is also applicable to other types of models such as machine learning models, AI models, and the like. Referring to FIG. **3**A, a host platform **320**, may host an IDE **310** (integrated development environment) where GenAI models, machine learning models, AI models, and the like may be developed, trained, retrained, and the like. In this example, the IDE **310** may include a software application with a user interface accessible by a user device over a network or through a local connection. For example, the IDE **310** may be embodied as a web application that can be accessed at a network address, URL, etc., by a device. As another example, the IDE **310** may be locally or remotely installed on a computing device used by a user.

(35) The IDE **310** may be used to design a model (via a user interface of the IDE), such as a generative artificial intelligence model that can receive text as input and generate custom imagery, etc. The model can then be executed/trained based on the training data established via the user interface. For example, the user interface may be used to build a new model. The training data for training such a new model may be provided from a training data store such as a database **324**, which includes training samples from the web, from customers, and the like. As another example, the training data may be pulled from one or more external data stores **330**, such as publicly available sites, etc.

(36) During training, the GenAI model **322** may be executed on training data via an AI engine **321** of the host platform **320**. The training data may include a large corpus of generic images and text that is related to those images. In the example embodiments, the training data may include software architecture diagrams (images) paired with descriptions (text) of the software architecture diagrams. The GenAI model **322** may learn mappings/connections between text and imagery during the execution and can thus create diagrams of the software architecture from input text. When the model is fully trained, it may be stored within the model repository **323** via the IDE **310** or the like.

(37) As another example, the IDE **310** may be used to retrain the GenAI model **322** after the model has already been deployed. Here, the training process may use executional results that have already been generated/output by the GenAI model **322** in a live environment (including any customer feedback, etc.) to retrain the GenAI model **322**. For example, predicted outputs/images that are custom generated by the GenAI model **322**, and the user feedback of the images may be used to retrain the model to enhance further the images that are generated for all users. The responses may include indications of whether the generated software architecture diagram is correct and, if not, what aspects of the diagram are incorrect. This data may be captured and stored within a runtime

log **325** or other data store within the live environment and can be subsequently used to retrain the GenAI model **322**.

(38) FIG. **3**B illustrates a process **300**B of executing a training process for training/retraining the GenAI model **322** via an AI engine **321**. In this example, a script **326** (executable) is developed and configured to read data from a database **324** and input the data to the GenAI model **322** while the GenAI model is running/executing via the AI engine **321**. For example, the script **326** may use identifiers of data locations (e.g., table IDs, row IDs, column IDs, topic IDs, object IDs, etc.) to identify locations of the training data within the database **324** and query an API **328** of the database **324**. In response, the database **324** may receive the query, load the requested data, and return it to the AI engine **321**, where it is input to the GenAI model **322**. The process may be managed via a user interface of the IDE **310**, which enables a human-in-the-loop during the training process (supervised learning). However, it should also be appreciated that the system is capable of unsupervised learning as well.

(39) The script **326** may iteratively retrieve additional training data sets from the database **324** and iteratively input the additional training data sets into the GenAI model **322** during the execution of the model to continue to train the model. The script may continue the process until instructions within the script tell the script to terminate, which may be based on a number of iterations (training loops), total time elapsed during the training process, etc.

(40) FIG. **3**C illustrates a process **300**C of designing a new AI model via a user interface **340** according to example embodiments. As an example, the user interface **340** may be output as part of the software application which interacts with the IDE **310** shown in FIG. **3**A, however, embodiments are not limited thereto. Referring to FIG. **3**C, a user can use an input mechanism to make selections from a menu **342** shown on the left-hand side of the user interface **340** to add pieces to the model such as data components, model components, analysis components, etc., within a workspace **344** of the user interface **340**.

(41) In the example of FIG. **3**C, the menu **342** includes a plurality of graphical user interface (GUI) menu options which can be selected to drill down into additional components that can be added to the model design shown in the workspace **344**. Here, the GUI menu options include options for adding features such as neural networks, machine learning models, AI models, data sources, conversion processes (e.g., vectorization, encoding, etc.), analytics, etc. The user can continue to add features to the model and connect them using edges or other means to create a flow within the workspace **344**. For example, the user may add a node **346** to a diagram of a new model within the workspace **344**. For example, the user may connect the node **346** to another node in the diagram via an edge **348**, creating a dependency within the diagram. When the user is done, the user can save the model for subsequent training/testing.

(42) According to various embodiments, the GenAI model described herein may be trained based on custom-defined prompts that are designed to draw out specific attributes associated with a goal of a user. These same prompts may be output during the live execution of the GenAI model. For example, a user may input a description of a goal and possibly other attributes. The description/attributes can then be used by the GenAI model to generate a custom image that enables the user to visualize the goal. The prompts may be generated via prompt engineering that can be performed through the model training process, such as the model training process described above in the examples of FIGS. **3**A-**3**C.

(43) Prompt engineering is the process of structuring sentences (prompts) so that the GenAI model understands them. A prompt may include a description of a goal, such as a goal of purchasing a particular type of car. The prompt may also provide a color, year, make, and model of the car. All of this information may be input into the GenAI model and used to create a custom image of the goal to enable the user to visualize the goal. Part of the prompting process may include delays/waiting times that are intentionally included within the script such that the model has time to think/understand the input data.

(44) FIG. **4** illustrates a process **400** of a GenAI model **422** generating an architecture diagram **424** of a software architecture based on prompts and responses to the prompts according to example embodiments. Referring to FIG. **4**, the GenAI model **422** may be hosted by a host platform and may be part of a software application **420** that is also hosted on the host platform. Here, the software application **420** may establish a connection with a user device **410**, such as a secure network connection. The secure connection may include a PIN, biometric scan, password, username, TTL handshake, etc.
(45) In the example of FIG. **4**, the software application **420** may control the interaction of the GenAI model **422** on the host platform and the user device **410**. In this example, the software application **420** may output queries on a user interface **412** of the user device **410** with requests for information from the user. The user may enter values into the fields on the user interface corresponding to the queries, and submit/transfer the data to the software application **420**, for example, by pressing a submit button, etc. In this example, the application may combine the query with the response from the user interface and generate a prompt that is submitted to the GenAI model **422**. For example, each prompt may include a combination of a query on the UI plus the response from the user. For example, if the query is "Please describe the purpose of the software system" and the response is "The software system is an API that manages access to a cloud database," then the text from both the prompt and the response to the prompt may be submitted to the GenAI model **422**.
(46) In some embodiments, the software application **420** may deliberately add waiting times between submitting prompts to the GenAI model **422** to ensure that the model has enough time to "think" about the answer. The waiting times may be integrated into the code of the software application **420**, or they may be modified/configured via a user interface. Furthermore, the ordering of the prompts and the follow-up questions that are asked may be different depending on the answers given during the previous prompt or prompts. The content within the prompts and the ordering of the prompts can cause the GenAI model **422** can architecture diagrams, descriptions of architecture diagrams, combinations of architecture diagrams, new architecture diagrams, and the like. Each prompt may include multiple components, including one or more of context, an instruction, input data, and an expected response/output.
(47) Software architecture diagrams provide a visual representation of the structure, relationships, and interactions within a software system. However, diagrams can be interpreted differently by different individuals and often lack details. Furthermore, it can be challenging to ensure that a codebase reflects the designed architecture. Also, most diagrams represent a static system view that may not capture runtime interactions adequately. Additionally, diagrams need to be updated regularly to reflect changes in the architecture, and keeping diagrams synchronized with the actual system can become a challenge.
(48) To mitigate these shortcomings, a generative artificial intelligence (AI) model can be trained on a repository of software architecture descriptions, including both graphics and text. This enables the model to learn a correlation between text and diagram components. The trained model can be used to generate a diagram of a software architecture from varying types of input and varying types of training.
(49) FIGS. **5**A-**5**D illustrate a process of generating a complex architecture diagram from a plurality of architectural views according to example embodiments. For example, FIG. **5**A illustrates a process **500** of generating a multi-domain architecture diagram **532** from multiple architecture "views" from different domains within an organization. In this example, a GenAI model **530** may receive architecture "views" from multiple different domains of the software architecture and generate a diagram (i.e., a complete view) of the multiple domains combined together. As such, the system can create a complete architecture of an organization, including multiple different domains and architects using GenAI.
(50) In this example, there are four different software architects, including architect **501**, architect

502, architect **503**, an architect **504**. The architects may use computing devices **511**, **512**, **513**, and **514**, respectively, to submit an architecture view **521**, an architecture view **522**, an architecture view **523**, and an architecture view **524** of different domains of the software architecture. A domain may refer to a software system, such as an email system that uses a shared database, an API, etc. An architect may be tasked with managing this system, while another architect is tasked with managing a separate software system such as a mobile application. Here, the GenAI model may receive the architecture views **521**, **522**, **523**, and **524** and generate the multi-domain architecture diagram **532**. Here, the multi-domain architecture diagram **532** may be in a JavaScript Object Notation (JSON) format, an extensible Markup Language (XML) format, a VISIO® format, and the like.

(51) In these examples, an architecture "view" may refer to a diagram of an architecture of a particular software application, a diagram from the perspective of a particular architect, from a particular domain, etc. Meanwhile, an architecture "diagram" may include a visualization of multiple architecture views, including intermediate software systems that are needed to glue the different domains together within the architecture diagram. Some examples of the architecture components that may be included within an architecture diagram include an executive summary, high-level requirements, architecture and design assumptions, application architecture, sequence diagrams, infrastructure architecture, security architecture, data architecture, legal and regulatory policy data and considerations, and the like.

(52) FIG. **5**B illustrates further details of the architecture views **521**, **522**, **523**, and **524**. In this example, each of the architectural views includes one or more architectural components. For example, the architecture view **521**, an internet banking software system that includes a web application, a single page application (e.g., a web application that interacts with a user and dynamically rewrites the current web page with new data, etc.), and a mobile application. Meanwhile, the architecture view **522** includes a diagram of a database component and an API that is used to access the database. The architecture view **523** includes an email system, and the architecture view **524** includes a mainframe banking system. These systems are just examples of the types of elements that may be present within a software architecture diagram and are not meant to be limiting the scope of the present application.

(53) FIG. **5**C illustrates a process **540** of generating the architecture diagram **532** from the architecture views **521**, **522**, **523**, and **524**. In this example, the GenAI model **530** may create edges **541**, **542**, **543**, **544**, **545**, **546**, and **547** between different domains/views of the software architecture and add a customer **548** to the diagram. Furthermore, annotations can be added to the edges to provide additional details, including actions, events, data transfers, etc., between two of the components within the diagram. The GenAI model **530** may "fit" together the different views of the architecture using generative artificial intelligence. The knowledge may be learned by training the GenAI model on a large corpus of software architecture documents that include both text and images or graphics related to the text.

(54) FIG. **5**D illustrates a process **550** of detecting a gap that exists within the multi-domain architecture diagram **532**. In this example, the user is viewing the multi-domain architecture diagram **532** on a user interface, such as a viewer of a software application or the like. Referring to FIG. **5**D, the GenAI model **530** may identify that the email system in a first domain is not in alignment with a web application in a second domain based on an out-of-date API. In response, the GenAI model **530** may generate and display a notification **552** with a description of the component that needs to be added to the system to make it in alignment.

(55) As another example, the GenAI model **530** may identify that a mainframe banking system in a third domain is missing an API for a database in a fourth domain of the software architecture. Here, the GenAI model **530** may output another notification **554** on the user interface, which identifies the missing API. In addition to outputting the notifications **552** and **554**, the GenAI model may identify the API and even display the name of the API as well as a link to a download location, etc.

(56) One example of how gaps may be identified is by looking at high-level requirements within a software architecture component and assessing whether additional information in the document is consistent. For example, the model may check to ensure that high-level requirements are shared (i.e., SSO must leverage the same SSO platform). Another example may be to identify misalignment to an organization's architecture principles (e.g., if most blueprints use a particular cloud service offering, but your new solution proposes using a different provider, the tool may recommend changing to the standard as identified by the majority of the documents, etc.) Furthermore, the model may also be trained on recommended patterns from external sources to allow for best practice recommendations as well as approved pattern repositories.

(57) FIG. **6** illustrates an example of a multimodal GenAI model **630** according to example embodiments. Referring to FIG. **6**, the multimodal GenAI model **630** may include multiple modes (or modalities) in which different processes are executed on different data in combination. For example, in FIG. **6**, a first modality **632** of the multimodal GenAI model **630** receives architecture views **610** and **620** and converts them into descriptive text and graphics (images) that correspond to the different architectural views. In some embodiments, the descriptive text may be a unified markup language (UML) diagram with both text and graphics or the like. Here, a second modality **634** of the multimodal GenAI model **630** receives the text and graphics **638** and generates an architecture diagram and stores it within a file such as an XML file, a JSON file, a VISIO® file, or the like.

(58) In the example of FIG. **6**, the first modality **632** processes images and generates text and graphics, which are fed into the second modality **634**. Here the second modality **634** generates the architecture diagram **640**, which includes both domains from the architecture views **610** and **620** within a single diagram. In some embodiments, the second modality **634** may receive input from a data store **636**, with architecture documentation and descriptions. The multimodal GenAI model **630** may learn from the text that is ingested for context, i.e., bigger picture value of architectural components and what capabilities they provide and learn from image data that is ingested for diagram representation. Potential to also leverage API marketplace to obtain API specifications and descriptions.

(59) Here, the architecture views may be converted into a text format which the model can more easily learn from. Furthermore, the GenAI model may prompt a user for additional information via a user interface (not shown), such as descriptions of architecture components, descriptions of service providers, etc., and use the responses along with the prompts to learn more information about the architecture further. Then, the model may generate the architecture diagram.

(60) FIGS. **7**A-**7**B illustrates processes for auditing a software architecture via a GenAI model according to example embodiments. Audits may commonly be performed on a software system to understand whether it satisfies corporate compliance policies and other best practices in the industry. The audits may be performed by internal auditors (who are part of the organization) and external auditors (who may require documentation of some kind).

(61) FIG. **7**A illustrates a process **700** in which a user may input a natural language query (e.g., such as a question about a software architecture of an organization, etc.) to a user interface **712** on a user device **710** and sends the query to a host platform **720** over a network. Here, the host platform **720** may host an application that includes the user interface **712**. In response, the host platform **720** may execute a GenAI model **722** on the query to generate a text-based response **724**. The text-based response **724** may include a descriptive answer to the user's query. For example, the query may include, please tell me the location of an API. In response, the GenAI model may identify the API within the architecture and generate a description of the functionality and the purpose of the API. The response may be displayed on the user interface **712** or output via an audio means or the like.

(62) FIG. **7**B illustrates a process **740**, which is like the process **700** that is performed in FIG. **7**A. However, in this example, the GenAI model **722** receives a text-based input from a user interface

732 of a user device **730** and generates an image-based response **726** (e.g., an answer, etc.). The image-based response **726** may include a diagram of the architecture component that the user is asking about or a diagram of another architecture component, etc. Here, the GenAI model is trained to identify correlations between text and architecture diagrams and can convert the text into an image.

(63) FIG. **8** illustrates a process **800** of building a software architecture diagram **840** based on conversation data according to example embodiments. In this example, the system may include a recorder that is in a central environment and that is able to record or at least transcribe and store conversations between users, such as calls, meetings, emails, texts, and the like. The recorder may be a software program that creates a log of the communications sent from the device (e.g., email, phone call, text message, etc.) and the content within the communications. As an example, the recorder may capture calls and/or texts **810** exchanged between a user **801** and a user **802** via their respective mobile devices. As another example, the recorder may capture emails and meetings **820** that are sent/conducted between a user **803** and a user **804** on their respective computing devices. The log may be input to a GenAI model **830**.

(64) In this example, the GenAI model **830** may collate relevant emails, calls, team messages, and meeting recordings to learn architectural components and what capabilities they provide and ingest images for diagram representation. Furthermore, the system may obtain documents created, written and verbal context attained through the respective mediums. The model may be multi-modal in order to process both images and text.

(65) FIG. **9** illustrates a process **900** of generating a "real-time" software architecture diagram using runtime data from systems of the software architecture according to example embodiments. As previously noted, software architecture documents are often "out of date" and refer to older versions or different versions of a software architecture. There are multiple reasons for this, including that generating diagrams can be challenging, and it can also be very time-consuming. In the example of FIG. **9**, runtime data from a plurality of systems **902**, **904**, **906**, and **908** is logged based on execution of the plurality of systems **902**, **904**, **906**, and **908** and the software that they host.

(66) GenAI model **920** generates an architecture diagram **930** of a software architecture based on runtime data (e.g., log data, etc.) from the plurality of systems **902**, **904**, **906**, and **908**. Here, the plurality of systems **902**, **904**, **906**, and **908** include one or more web servers, cloud platforms, on-premises servers, databases, software applications, APIs, etc. when executing during live operation.

(67) In response to receiving the runtime data, the GenAI model **920** can generate a diagram of the architecture based on the words included in the runtime data because the GenAI model **920** learns a correlation between text and images of architecture documents. To do this, the GenAI model **920** is trained on a large corpus of architectural documents and images. Here, the log data includes text data that can be interpreted by the GenAI model **920** to generate a corresponding architecture diagram that is based on the current runtime data of the system and is therefore considered "real-time."

(68) FIG. **10**A illustrates a method **1000** of generating a real-time diagram of a software architecture according to example embodiments. As an example, the method **1000** may be performed by a computing system, a software application, a server, a cloud platform, a combination of systems, and the like. Referring to FIG. **10**A, in **1001**, the method may include storing software architecture diagrams in a data store. In **1002**, the method may include receiving runtime data from a plurality of different software systems within a software architecture; the runtime data may include descriptions of events that occur during runtime between the different software systems of the software architecture. In **1003**, the method may include generating a diagram of the software architecture based on the execution of a multi-modal generative artificial intelligence (GenAI) model on the runtime data and the software architecture diagrams stored in the data store. In **1004**, the method may include displaying the diagram of the software architecture via a user interface.

(69) In some embodiments, the generating may include generating a unified markup language (UML) diagram via the GenAI model, wherein the UML diagram may include both text and graphics that describe the plurality of different architecture diagrams of the software architecture. In some embodiments, the generating the description may include generating the diagram of the software architecture based on the UML diagram and based on the stored software architecture documents in the data store via the second process of the GenAI model.

(70) In some embodiments, the runtime data may include one or more of calls, data flows, and communications between a plurality of different components of the software architecture, and the generating may include fitting the plurality of different components together to generate a complete diagram of the plurality of different pieces of the software architecture based on the execution of the GenAI model. In some embodiments, the method may further include receiving feedback about the generated diagram of the software architecture via the user interface and retraining the GenAI model based on the generated diagram and the received feedback.

(71) In some embodiments, the generating the diagram of the software architecture may include generating the diagram of the software architecture via a software application and rendering the diagram of the software architecture via a viewer of the software application, which is displayed on the user interface. In some embodiments, the receiving may further include receiving descriptions of a plurality of application programming interfaces (APIs) of the software architecture, and the generating may further include generating the diagram of the software architecture based on the execution of the GenAI on the descriptions of the plurality of APIs. In some embodiments, the generating may further include displaying a prompt on the user interface, receiving a response to the prompt via the user interface, and generating the diagram of the software architecture based on the prompt and the response to the prompt.

(72) FIG. **10**B illustrates a method **1010** of generating a multi-domain architecture diagram from multiple domain views according to example embodiments. As an example, the method **1010** may be performed by a computing system, a software application, a server, a cloud platform, a combination of systems, and the like. Referring to FIG. **10**B, in **1011**, the method may include training a generative artificial intelligence (GenAI) model via execution of the GenAI model on descriptions and diagrams of a software architecture. In **1012**, the method may include receiving a plurality of different architectural views from a plurality of different domains of the software architecture. In **1013**, the method may include generating an architecture diagram of the plurality of domains of the software architecture in combination based on the execution of the GenAI model on the plurality of different architecture views from the plurality of different domains. In **1014**, the method may include displaying the architecture diagram of the plurality of domains of the software architecture via a user interface.

(73) In some embodiments, the receiving may include receiving a plurality of different blueprints of the plurality of different domains of the software architecture, and the generating may include generating the architecture diagram based on the execution of the GenAI model on the plurality of different blueprints. In some embodiments, the receiving may include receiving the plurality of different architecture views from a plurality of different software architects, and the generating may include generating the architecture diagram based on the execution of the GenAI model on the plurality of different architecture views from a plurality of different software architects. In some embodiments, the method may further include receiving feedback about the architecture diagram via the user interface and retraining the GenAI model based on the architecture diagram and the received feedback on the architecture diagram.

(74) In some embodiments, the method may further include generating suggested changes to the software architecture via the execution of a machine learning model on the architecture diagram of the software architecture. In some embodiments, the method may further include receiving a feature set that may include a description of features for a new software architecture and generating an architecture diagram of the new software architecture via execution of the GenAI model on the

feature set. In some embodiments, the generating may further include displaying a prompt on the user interface, receiving a response to the prompt via the user interface, and generating the architecture diagram of the plurality of domains based on the prompt and the response to the prompt.

(75) In some embodiments, the GenAI model may include a multi-modal GenAI model, and the generating may include generating a description of the software architecture based on the plurality of different architecture diagrams via a first modality of the multi-modal GenAI model and generating the architecture diagram of the plurality of domains of the software architecture based on the description via a second modality of the multi-modal GenAI model.

(76) FIG. **10**C illustrates a method **1020** of identifying gaps in a software architecture according to example embodiments. As an example, the method **1020** may be performed by a computing system, a software application, a server, a cloud platform, a combination of systems, and the like. Referring to FIG. **10**C, in **1021**, the method may include receiving a plurality of architecture documents of a plurality of different domains of a software architecture. In **1022**, the method may include identifying a missing component that is missing from within the software architecture between a first domain and a second domain among the plurality of domains based on the execution of a generative artificial intelligence (GenAI) model based on the plurality of architecture documents of the software architecture. In **1023**, the method may include generating a recommended modification to the software architecture based on the identified missing component. In **1024**, the method may include displaying the recommended modification via a user interface.

(77) In some embodiments, the identifying may include identifying the missing component based on a misalignment between service providers of the first and second domains detected by the GenAI model from the plurality of architecture documents. In some embodiments, the identifying may include identifying the missing component based on a difference in requirements between the first and second domains detected by the GenAI model based on the plurality of architecture documents. In some embodiments, the identifying may include executing the GenAI model on best practice architecture documents and identifying the missing component based on a component within the software architecture that the GenAI model detects as not being best practice based on the plurality of architecture documents.

(78) In some embodiments, the identifying may include identifying a location within the software architecture for a new application programming interface (API) between a software component in the first domain and another software component in the second domain by the GenAI model based on the plurality of architecture documents. In some embodiments, the generating may include generating a description of the missing component via the GenAI model based on architecture diagrams of the first and second domains from among the plurality of architecture documents. In some embodiments, the method may further include training the GenAI model based on the execution of the GenAI model on descriptions of software applications within the software architecture, architecture blueprints of the software architecture, and security policies associated with the software architecture. In some embodiments, the method may further include generating a prompt with a request for information about the software by the GenAI model based on the plurality of architecture documents and displaying the prompt on the user interface.

(79) FIG. **10**D illustrates a method **1030** of auditing a software architecture via a GenAI model according to example embodiments. As an example, the method **1030** may be performed by a computing system, a software application, a server, a cloud platform, a combination of systems, and the like. Referring to FIG. **10**D, in **1031**, the method may include training a generative artificial intelligence (GenAI) model based on architecture diagrams of a software architecture and descriptions of the architecture diagrams. In **1032**, the method may include displaying one or more prompts on a user interface. In **1033**, the method may include receiving one or more natural language responses associated with the software architecture in response to the one or more prompts. In **1034**, the method may include generating a text-based response to the natural language

query submitted via the user interface based on the execution of the GenAI model on the one or more prompts and the one or more natural language responses associated with the software architecture. In **1035**, the method may include displaying the text-based response via the user interface.

(80) In some embodiments, the method may further include generating a diagram that illustrates a portion of the software architecture based on the text-based response and displaying the diagram with the text-based response via the user interface. In some embodiments, the GenAI model may include a multi-modal model, and the generating may include generating the text-based response via a first mode of the multi-modal model and generating the diagram via a second mode of the multi-modal model. In some embodiments, the receiving may include receiving a question about the software architecture from a user and in response, generating an answer to the question based on execution of the GenAI model on the question and the one or more prompts and displaying the answer via the user interface.

(81) In some embodiments, the training may include training the GenAI model based on the execution of the GenAI model on blueprints of the software architecture and descriptions of the software architecture. In some embodiments, the receiving may include receiving a request for a view of the software architecture, and the generating may further include generating a diagram of the view of the software architecture and displaying the diagram of the view via the user interface. In some embodiments, the method may further include receiving feedback about the text-based response and retraining the GenAI model based on the text-based response and the feedback to the text-based response. In some embodiments, the generating may further include receiving a runtime data from the software architecture and generating the text-based response to the natural language query based on the execution of the GenAI model on the runtime data.

(82) FIG. **10**E illustrates a method **1040** of generating a diagram of a software architecture based on conversation data according to example embodiments. As an example, the method **1040** may be performed by a computing system, a software application, a server, a cloud platform, a combination of systems, and the like. Referring to FIG. **10**E, in **1041**, the method may include receiving and recording electronic communications that occur between users of an organization within a data store. In **1042**, the method may include receiving an input via a user interface. In **1043**, the method may include retrieving the electronic communications of the organization and software architecture documents from the data store in response to receipt of the input. In **1044**, the method may include generating a diagram of a software architecture of the organization based on the execution of a generative artificial intelligence (GenAI) model on the electronic communications of the organization and the software architecture documents. In **1045**, the method may include displaying the diagram of the software architecture via a user interface.

(83) In some embodiments, the method may further include training the GenAI model to generate the diagram based on the execution of the GenAI model on descriptions and images of software architectures. In some embodiments, the capturing may include capturing electronic communications, including one or more of an electronic mail, a phone call, and a meeting, and recording content from the captured electronic communications in the data store. In some embodiments, the method may further include validating the diagram of the software architecture based on a predefined validation model and displaying the results of the validation via the user interface.

(84) In some embodiments, the method may further include generating a prompt with a request for information about the software architecture by the GenAI model based on the software architecture documents and displaying the prompt on the user interface. In some embodiments, the method may further include receiving a response to the prompt, and the generating may include generating the diagram of the software architecture of the organization based on the prompt and the response to the prompt. In some embodiments, the generating the diagram of the software architecture of the organization may include generating an image within a format of the software application, and the

displaying may include displaying the image via a viewer of the software application within the user interface.

(85) The above embodiments may be implemented in hardware, in a computer program executed by a processor, in firmware, or in a combination of the above. A computer program may be embodied on a computer-readable medium, such as a storage medium. For example, a computer program may reside in random access memory ("RAM"), flash memory, read-only memory ("ROM"), erasable programmable read-only memory ("EPROM"), electrically erasable programmable read-only memory ("EEPROM"), registers, hard disk, a removable disk, a compact disk read-only memory ("CD-ROM"), or any other form of storage medium known in the art.

(86) An exemplary storage medium may be coupled to the processor such that the processor may read information from, and write information to, the storage medium. In the alternative, the storage medium may be integral to the processor. The processor and the storage medium may reside in an application-specific integrated circuit ("ASIC"). In the alternative, the processor and the storage medium may reside as discrete components. For example, FIG. **11***g* illustrates an example computer system architecture, which may represent or be integrated in any of the above-described components, etc.

(87) FIG. **11** illustrates an example system **1100** that supports one or more of the example embodiments described and/or depicted herein. The system **1100** comprises a computer system/server **1102**, which is operational with numerous other general-purpose or special-purpose computing system environments or configurations. Examples of well-known computing systems, environments, and/or configurations that may be suitable for use with computer system/server **1102** include, but are not limited to, personal computer systems, server computer systems, thin clients, thick clients, hand-held or laptop devices, multiprocessor systems, microprocessor-based systems, set-top boxes, programmable consumer electronics, network PCs, minicomputer systems, mainframe computer systems, and distributed cloud computing environments that include any of the above systems or devices, and the like.

(88) Computer system/server **1102** may be described in the general context of computer system-executable instructions, such as program modules, being executed by a computer system. Generally, program modules may include routines, programs, objects, components, logic, data structures, and so on that perform particular tasks or implement particular abstract data types. Computer system/server **1102** may be practiced in distributed cloud computing environments where tasks are performed by remote processing devices that are linked through a communications network. In a distributed cloud computing environment, program modules may be located in both local and remote computer system storage media, including memory storage devices.

(89) As shown in FIG. **11**, computer system/server **1102** in the example system **1100** is shown in the form of a general-purpose computing device. The components of computer system/server **1102** may include, but are not limited to, one or more processors or processing units (processor **1104**), a system memory **1106**, and a bus that couples various system components, including the system memory **1106** to the processor **1104**.

(90) The bus represents one or more of any of several types of bus structures, including a memory bus or memory controller, a peripheral bus, an accelerated graphics port, and a processor or local bus using any of a variety of bus architectures. By way of example, and not limitation, such architectures include Industry Standard Architecture (ISA) bus, Micro Channel Architecture (MCA) bus, Enhanced ISA (EISA) bus, Video Electronics Standards Association (VESA) local bus, and Peripheral Component Interconnects (PCI) bus.

(91) Computer system/server **1102** typically includes a variety of computer system-readable media. Such media may be any available media that is accessible by computer system/server **1102**, and it includes both volatile and non-volatile media, removable and non-removable media. The system memory **1106**, in one embodiment, implements the flow diagrams of the other figures. The system memory **1106** can include computer system readable media in the form of volatile memory, such as

random-access memory (RAM) **1110** and/or cache memory **1112**. Computer system/server **1102** may further include other removable/non-removable, volatile/non-volatile computer system storage media. By way of example only, storage system **1114** can be provided for reading from and writing to a non-removable, non-volatile magnetic media (not shown and typically called a “hard drive”). Although not shown, a magnetic disk drive for reading from and writing to a removable, non-volatile magnetic disk (e.g., a “floppy disk”), and an optical disk drive for reading from or writing to a removable, non-volatile optical disk such as a CD-ROM, DVD-ROM or other optical media can be provided. In such instances, each can be connected to the bus by one or more data media interfaces. As will be further depicted and described below, the system memory **1106** may include at least one program product having a set (e.g., at least one) of program modules that are configured to carry out the functions of various embodiments of the application.

(92) Program/utility **1116**, having a set (at least one) of program modules **1118**, may be stored in the system memory **1106** by way of example, and not limitation, as well as an operating system, one or more application programs, other program modules, and program data. Each of the operating systems, one or more application programs, other program modules, and program data or some combination thereof may include an implementation of a networking environment. Program modules **1118** generally carry out the functions and/or methodologies of various embodiments of the application as described herein.

(93) As will be appreciated by one skilled in the art, aspects of the present application may be embodied as a system, method, or computer program product. Accordingly, aspects of the present application may take the form of an entirely hardware embodiment, an entirely software embodiment (including firmware, resident software, micro-code, etc.), or an embodiment combining software and hardware aspects that may all generally be referred to herein as a “circuit,” “module” or “system.” Furthermore, aspects of the present application may take the form of a computer program product embodied in one or more computer-readable medium(s) having computer-readable program code embodied thereon.

(94) Computer system/server **1102** may also communicate with one or more external devices **1120** such as a keyboard, a pointing device, a display **1122**, etc.; one or more devices that enable a user to interact with computer system/server **1102**; and/or any devices (e.g., network card, modem, etc.) that enable computer system/server **1102** to communicate with one or more other computing devices. Such communication can occur via I/O interfaces **1124**. Still yet, computer system/server **1102** can communicate with one or more networks, such as a local area network (LAN), a general wide area network (WAN), and/or a public network (e.g., the Internet) via network adapter **1126**. As depicted, network adapter **1126** communicates with the other components of computer system/server **1102** via a bus. Although not shown, other hardware and/or software components could be used in conjunction with the computer system/server **1102**. Examples include but are not limited to microcode, device drivers, redundant processing units, external disk drive arrays, RAID systems, tape drives, and data archival storage systems, etc.

(95) Although an exemplary embodiment of at least one of a system, method, and computer-readable medium has been illustrated in the accompanying drawings and described in the foregoing detailed description, it will be understood that the application is not limited to the embodiments disclosed but is capable of numerous rearrangements, modifications, and substitutions as set forth and defined by the following claims. For example, the system's capabilities of the various figures can be performed by one or more of the modules or components described herein or in a distributed architecture and may include a transmitter, receiver, or pair of both. For example, all or part of the functionality performed by the individual modules may be performed by one or more of these modules. Further, the functionality described herein may be performed at various times and in relation to various events, internal or external to the modules or components. Also, the information sent between various modules can be sent between the modules via at least one of: a data network, the Internet, a voice network, an Internet Protocol network, a wireless device, a wired device and/or

via a plurality of protocols. Also, the messages sent or received by any of the modules may be sent or received directly and/or via one or more of the other modules.
(96) One skilled in the art will appreciate that a “system” could be embodied as a personal computer, a server, a console, a personal digital assistant (PDA), a cell phone, a tablet computing device, a smartphone, or any other suitable computing device, or combination of devices. Presenting the above-described functions as being performed by a “system” is not intended to limit the scope of the present application in any way but is intended to provide one example of many embodiments. Indeed, methods, systems, and apparatuses disclosed herein may be implemented in localized and distributed forms consistent with computing technology.
(97) It should be noted that some of the system features described in this specification have been presented as modules to more particularly emphasize their implementation independence. For example, a module may be implemented as a hardware circuit comprising custom very large-scale integration (VLSI) circuits or gate arrays, off-the-shelf semiconductors such as logic chips, transistors, or other discrete components. A module may also be implemented in programmable hardware devices such as field programmable gate arrays, programmable array logic, programmable logic devices, graphics processing units, or the like.
(98) A module may also be at least partially implemented in software for execution by various types of processors. An identified unit of executable code may, for instance, comprise one or more physical or logical blocks of computer instructions that may, for instance, be organized as an object, procedure, or function. Nevertheless, the executables of an identified module need not be physically located together but may comprise disparate instructions stored in different locations, which, when joined logically together, comprise the module and achieve the stated purpose for the module. Further, modules may be stored on a computer-readable medium, which may be, for instance, a hard disk drive, flash device, random access memory (RAM), tape, or any other such medium used to store data.
(99) Indeed, a module of executable code could be a single instruction or many instructions and may even be distributed over several different code segments, among different programs, and across several memory devices. Similarly, operational data may be identified and illustrated herein within modules and may be embodied in any suitable form and organized within any suitable type of data structure. The operational data may be collected as a single data set or may be distributed over different locations, including over different storage devices, and may exist, at least partially, merely as electronic signals on a system or network.
(100) It will be readily understood that the application components, as generally described and illustrated in the figures herein, may be arranged and designed in a wide variety of configurations. Thus, the detailed description of the embodiments is not intended to limit the scope of the application as claimed but is merely representative of selected embodiments of the application.
(101) One having ordinary skill in the art will readily understand that the above may be practiced with steps in a different order and/or with hardware elements in configurations that are different from those which are disclosed. Therefore, although the application has been described based upon these preferred embodiments, certain modifications, variations, and alternative constructions would be apparent to those of skill in the art.
(102) While preferred embodiments of the present application have been described, it is to be understood that the embodiments described are illustrative only. The scope of the application is to be defined solely by the appended claims when considered with a full range of equivalents and modifications (e.g., protocols, hardware devices, software platforms, etc.) thereto.

Claims

1. An apparatus comprising: a data store comprising software architecture documents; and at least one processor configured to: train a generative artificial intelligence (GenAI) model with a neural

network capability to generate software architecture diagrams based on execution of the GenAI model on blueprints of historical software architecture documents and communication content, receive and record electronic communications that occur within an organization in the data store, receive an input via a user interface, generate a diagram of a software architecture of the organization based on execution of the trained GenAI model on the electronic communications, and display the diagram of the software architecture via the user interface.

2. The apparatus of claim 1, wherein the at least one processor is configured to record at least one of an electronic mail, a phone call, and a meeting, and store content from the electronic communications recorded in the data store.

3. The apparatus of claim 1, wherein the at least one processor is further configured to validate the diagram of the software architecture based on a predefined validation model and display validation results of the diagram via the user interface.

4. The apparatus of claim 1, wherein the at least one processor is further configured to generate a prompt with a request for information about the software architecture by the GenAI model, and display the prompt on the user interface.

5. The apparatus of claim 4, wherein the at least one processor is further configured to receive a response to the prompt, and generate the diagram of the software architecture based on the prompt and the response to the prompt.

6. The apparatus of claim 1, wherein the at least one processor is configured to generate an image within a format of a software application, and the display the image via a viewer of the software application within the user interface.

7. The apparatus of claim 1, wherein the at least one processor is configured to generate a digital document with the diagram of the software architecture, wherein the digital document comprises descriptions and images of an architecture of at least one software system used by the organization.

8. A method comprising: training a generative artificial intelligence (GenAI) model with a neural network capability on to generate software architecture diagrams based on execution of the GenAI model on blueprints of historical software architecture documents and communication content; receiving and recording electronic communications that occur within an organization in a data store; receiving an input via a user interface; generating a diagram of a software architecture based on execution of the trained GenAI model on the electronic communications; and displaying the diagram of the software architecture via the user interface.

9. The method of claim 8, wherein the capturing receiving comprises capturing electronic communications including at least one of an electronic mail, a phone call, and a meeting, and recording content from the captured electronic communications in the data store.

10. The method of claim 8, wherein the method further comprises validating the diagram of the software architecture based on a predefined validation model and displaying validation results of the diagram via the user interface.

11. The method of claim 8, wherein the method further comprises generating a prompt with a request for information about the software architecture by the GenAI model, and displaying the prompt on the user interface.

12. The method of claim 11, wherein the method further comprises receiving a response to the prompt, wherein the generating further comprises generating the diagram of the software architecture based on the prompt and the response to the prompt.

13. The method of claim 8, wherein the generating the diagram of the software architecture of the organization comprises generating an image within a format of a software application, and the displaying comprises displaying the image via a viewer of the software application within the user interface.

14. A non-transitory computer-readable medium comprising instructions stored therein which, when executed by a processor, cause the processor to perform: training a generative artificial intelligence (GenAI) model with a neural network capability to generate software architecture

diagrams based on execution of the GenAI model on blueprints of historical software architecture documents and communication content; receiving and recording electronic communications that occur within an organization in a data store; receiving an input via a user interface; generating a diagram of a software architecture of the organization based on execution of the GenAI model on the electronic communications; and displaying the diagram of the software architecture via the user interface.

15. The non-transitory computer-readable medium of claim 14, wherein the processor is further configured to perform validating the diagram of the software architecture based on a predefined validation model and displaying validation results of the diagram via the user interface.

16. The non-transitory computer-readable medium of claim 14, wherein the processor is further configured to perform generating a prompt with a request for information about the software architecture by the GenAI model, and displaying the prompt on the user interface.

17. The non-transitory computer-readable medium of claim 16, wherein the processor is further configured to perform receiving a response to the prompt, wherein the generating further comprises generating the diagram of the software architecture based on the prompt and the response to the prompt.